SMART QUEUE AND TOKEN MANAGEMENT SYSTEM

A PROJECT REPORT

Submitted by

SAISANJAY SS

240701458

in partial fulfilment of the End Semester Examination for the course

CB23F35 –
LOW-CODE APPLICATION DEVELOPMENT USING ZOHO CREATOR

in

BACHELOR OF ENGINEERING
COMPUTER SCIENCE AND ENGINEERING



RAJALAKSHMI ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)

Rajalakshmi Nagar, Thandalam, Tamil Nadu – 602 105

NOVEMBER 2026

RAJALAKSHMI ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)

BONAFIDE CERTIFICATE

Certified that this project report titled "**SMART QUEUE AND TOKEN MANAGEMENT SYSTEM**" is the bonafide work of **SAI SANJAY [2116240701458]**, who carried out the individual project work on the Zoho Creator low-code platform for the course CB23F35 – Low-Code Application Development using Zoho Creator during the academic year [2026-2027], under the supervision of Dr. Jinu Sophia J.

SIGNATURE

Dr. Manoranjini J
HEAD OF THE DEPARTMENT
Department of Computer Science and Engineering
Rajalakshmi Engineering College

SIGNATURE

Dr. Jinu Sophia J
COURSE FACULTY
Associate Professor
Department of Computer Science and Engineering
Rajalakshmi Engineering College

Submitted for the End Semester Examination held on: _____

INTERNAL EXAMINER **EXTERNAL EXAMINER**

DECLARATION

I, SAISANJAY SS [2116240701458], hereby declare that the project report titled “Smart Queue and Token Management System” submitted for the End Semester Examination of the course CB23F35 – Low-Code Application Development using Zoho Creator, is a record of original work carried out by me individually. The Zoho Creator application described in this report – including its forms, lookup relationships, reports, dashboard, Deluge script and workflow – was designed and developed by me.

I further declare that this work has not been submitted, in part or in full, for the award of any other degree or course credit. Features that have been planned but not yet implemented and tested are identified as proposed functionality in this report.

Place: Chennai
Date:

SIGNATURE OF THE STUDENT
SAISANJAY SS
[2116240701458]

ACKNOWLEDGEMENT

I express my sincere gratitude to our Chairman Thiru. S. Meganathan, Chairperson Dr. (Mrs.) Thangam Meganathan and Vice-Chairman Mr. Abhay Meganathan for providing the infrastructure and facilities that made this work possible.

I am grateful to our Principal Dr. S.N. MURUGESAN, Rajalakshmi Engineering College, for the constant encouragement extended to students to take up industry-relevant learning.

I thank Dr. Manoranjini J, Head of the Department, Department of Computer Science and Engineering for the guidance and support provided throughout the course.

I owe my sincere thanks to my course faculty, Dr. Jinu Sophia J, Associate professor, for the valuable guidance, reviews and feedback that shaped this application and report.

I also acknowledge Zoho Creator Education Program for providing access to the Zoho Creator platform and learning resources, and I thank my family and friends for their unwavering support.

SAISANJAY SS

ABSTRACT

Queues at service counters are commonly managed with paper slips, verbal calling and handwritten registers. Customers do not know their position or expected waiting time, staff cannot see the whole queue, and no reliable record of service is kept. These weaknesses cause long waiting times, poor organisation and a poor customer experience.

The Smart Queue and Token Management System is a low-code application developed on Zoho Creator. It contains ten forms – Services, Customer_Registration, Token_Management, Queue_Settings, Staff_Management, Departments, Customer_Feedback, Queue_Activity_Log, Daily_Queue_Summary and Notifications – connected through lookup relationships. When a customer registration is submitted, a Deluge workflow counts the existing tokens, generates a token with the prefix "T", creates a Token_Management record with the status Waiting, assigns counter number 1 and records a Created entry in Queue_Activity_Log. The application provides a Waiting Queue report, an All Tokens report and a dashboard showing waiting and serving counts and tables.

The registration and token-generation workflow has been implemented and tested. Call Next, remaining status transitions, role-based permissions, notifications, feedback processing and daily summary automation are proposed functionality. The current record-count-based token numbering is suitable for a prototype but should be improved to prevent duplicate numbers.

Keywords: Low-Code Development; Zoho Creator; Deluge Scripting; Queue Management; Token Management; Workflow Automation.

PROJECT AT A GLANCE

Attribute	Details
Project Title	Smart Queue and Token Management System
Application Domain	Queue and token management for service counters
Student Name	Saisanjay SS
Register Number	2116240701458
Development Platform	Zoho Creator (low-code); Deluge for custom logic
Forms	15 forms (14 data-entry forms and 1 configuration form)
Lookup Relationships	Customer_Registration→Services; Token_Management→Customer_Registration; Queue_Activity_Log→Token_Management
Token Statuses	Waiting, Serving, Completed, Cancelled
Activity Types	Created, Called, Started, Completed, Cancelled
Implemented	Registration; automatic token; Waiting status; activity log on creation; Waiting Queue and All Tokens reports; dashboard
Proposed / Not confirmed	Call Next; other status transitions; role-based permissions; approval workflow; notifications; feedback processing; daily summary
Deployment Status	Development prototype; production deployment not verified

TABLE OF CONTENTS

CHAPTER 1 INTRODUCTION11

1.1 Background11
1.2 Introduction to Queue Management11
1.3 Need for the Proposed System11
1.4 Problem Statement11
1.5 Project Objectives11
1.6 Scope of the Project12
1.7 Organisation of the Report12

CHAPTER 2 LITERATURE SURVEY AND SYSTEM STUDY13

2.1 Existing Queue Management Approaches13
2.2 Manual Queue Management Systems13
2.3 Traditional Token Systems13
2.4 Digital Queue Management Systems13
2.5 Low-Code Development Platforms13
2.6 Introduction to Zoho Creator14
2.7 Existing System Limitations14
2.8 Proposed System14
2.9 Advantages of the Proposed System14

CHAPTER 3 REQUIREMENT ANALYSIS15

3.1 Functional Requirements15
3.2 Non-Functional Requirements15
3.3 Hardware Requirements15
3.4 Software Requirements16
3.5 User Roles16
3.6 Use Cases16
3.7 Feasibility Study16
3.8 Functional and Technical Constraints16

CHAPTER 4 SYSTEM DESIGN18

4.1 System Architecture18
4.2 Overall System Workflow18
4.3 Module Design18
4.4 Data Flow Diagrams19
4.5 Entity Relationship Diagram20
4.6 Database and Form Design20
4.7 Workflow Design21
4.8 Use Case Diagram21
4.9 Activity Diagrams21
4.10 System Navigation Structure22

CHAPTER 5 IMPLEMENTATION23

5.1 Development Environment23
5.2 Zoho Creator Application Development23
5.3 Form Creation23
5.4 Module Implementation23
5.5 Lookup Relationships24
5.6 Customer Registration Workflow24
5.7 Automatic Token Generation24
5.8 Deluge Code Explanation24
5.9 Token Status Management25
5.10 Reports and Dashboards26
5.11 Workflow Implementation Details26
5.12 Screenshots and Explanations27

CHAPTER 6 TESTING AND DEBUGGING28

6.1 Testing Approach28
6.2 Unit Testing28
6.3 Integration Testing28

6.4 Functional Testing28
6.5 Test Cases28
6.6 Error Handling29
6.7 Debugging29
6.8 Limitations29

CHAPTER 7 DEPLOYMENT, SECURITY AND GOVERNANCE30
7.1 Application Deployment Using Zoho Creator30
7.2 User Access Management30
7.3 Data Security30
7.4 Role-Based Access Concepts30
7.5 Data Validation30
7.6 Workflow Governance30
7.7 Privacy Considerations31
7.8 Backup and Maintenance31
7.9 Limitations of the Current Prototype31

CHAPTER 8 RESULTS AND DISCUSSION32
8.1 Application Output32
8.2 Customer Registration Output32
8.3 Generated Token Output32
8.4 Queue Management Reports32
8.5 Dashboard Output32
8.6 Evaluation Against Objectives33
8.7 Comparison with Traditional Queue Management33
8.8 Benefits33
8.9 Current Limitations33
8.10 Discussion33

CHAPTER 9 CONCLUSION AND FUTURE ENHANCEMENTS34
9.1 Project Conclusion34
9.2 Achievements34
9.3 Learning Outcomes34
9.4 Current Limitations34
9.5 Future Enhancements34
9.6 Final Remarks34

REFERENCES34

APPENDIX A FUNCTIONAL FEATURE COVERAGE35

APPENDIX B APPLICATION FORMS, REPORTS AND DASHBOARD PAGES35

APPENDIX C DELUGE CODE SUMMARY36

APPENDIX D USER MANUAL36

APPENDIX E TESTING AND EVALUATION TABLES37

APPENDIX F PROJECT EVALUATION / VIVA DOCUMENTATION37

LIST OF TABLES

Table 1.1: Project Objectives and Current Status
Table 2.1: Low-Code and Traditional Development Comparison
Table 3.1: Functional Requirements
Table 3.2: Non-Functional Requirements
Table 3.3: Hardware Requirements
Table 3.4: Software Requirements
Table 3.5: User Roles and Responsibilities
Table 3.6: Use Case Descriptions
Table 4.1: Module Design
Table 4.2: Lookup Relationships
Table 4.3: Fields Populated by the Registration Workflow
Table 4.4: Token Statuses and Activity Types
Table 5.1: Development Environment
Table 5.2: Line-by-Line Explanation of the Deluge Code
Table 5.3: Token Status Transitions
Table 5.4: Reports and Dashboard Implemented
Table 5.5: Workflow Summary
Table 5.6: Screenshot Checklist
Table 6.1: Test Cases – Expected and Actual Results
Table 7.1: Deployment Checklist
Table 7.2: Proposed Role-Based Access
Table 8.1: Objective Achievement Mapping
Table 8.2: Comparison with Traditional Queue Management
Table A.1: Feature Coverage
Table B.1: Lookup Relationships
Table C.1: Key Deluge Elements Used
Table D.1: Troubleshooting
Table E.1: Test Summary
Table E.2: Evaluation Rubric
Table F.1: Student and Project Details
Table F.2: Viva Preparation
Table F.3: Marks Summary

LIST OF FIGURES

Figure 2.1: Proposed queue and token process (dashed steps are proposed)
Figure 4.1: Logical architecture of the Smart Queue and Token Management System
Figure 4.2: Overall system workflow (dashed boxes are proposed)
Figure 4.3: Data Flow Diagram – Level 0
Figure 4.4: Data Flow Diagram – Level 1
Figure 4.5: Entity relationship diagram
Figure 4.6: Use case diagram (starred items are proposed)
Figure 4.7: Activity diagram – customer registration
Figure 4.8: Activity diagram – token processing
Figure 4.9: Application navigation structure
Figure 5.1: Smart Queue and Token Management application in Zoho Creator
Figure 5.2: List of application forms
Figure 5.3: Customer Registration form developed using Zoho Creator
Figure 5.4: Lookup field configuration
Figure 5.5: Customer registration workflow configuration
Figure 5.6: All Tokens report
Figure 5.7: Waiting Queue report
Figure 5.8: Queue activity log record created during token generation
Figure 5.9: Queue dashboard
Figure 8.1: Customer registration submission
Figure 8.2: Generated token record
Figure 8.3: Activity log entry for the new token

LIST OF ABBREVIATIONS

Abbreviation	Expansion
API	Application Programming Interface
BE	Bachelor of Engineering
CRUD	Create, Read, Update and Delete
DFD	Data Flow Diagram
ER	Entity–Relationship
LCAP	Low-Code Application Platform
RBAC	Role-Based Access Control
UI	User Interface
URL	Uniform Resource Locator

CHAPTER 1
INTRODUCTION

1.1 Background

Service-oriented organisations such as bank branches, hospital outpatient desks, government offices, college administrative sections and service centres must handle many customers who arrive without prior scheduling. When the number of customers exceeds the number of available service counters, a queue forms. If the queue is managed informally, customers do not know when their turn will come, staff cannot see how many customers are waiting, and management has no record of how the service was delivered.

The Smart Queue and Token Management System is a low-code application developed on the Zoho Creator platform. It replaces the physical line with a digital register: a customer is registered once, a unique token is generated automatically, and the token is tracked through its status until the service is completed or cancelled. Every token-related event is written to an activity log so that the queue history is available for review.

1.2 Introduction to Queue Management

Queue management is the discipline of organising the arrival, waiting and servicing of customers so that waiting time is minimised and service counters are used effectively. A token-based queue assigns each arriving customer a sequence identifier. The identifier, rather than the customer's physical position in a line, determines when service begins. A digital token system extends this idea by storing the token, its status and its timestamps in a database so that they can be displayed, searched and reported.

1.3 Need for the Proposed System

Manual or paper-based token handling is slow to administer and error-prone. Token slips are lost, numbers are repeated, and staff must rely on memory or handwritten registers to decide who is next. There is no consolidated view of how many customers are waiting or being served, and no reliable data from which management can analyse peak hours or service efficiency. A centralised digital system is therefore needed to automate token generation, show the live queue and retain service records.

1.4 Problem Statement

Traditional queue management systems often require customers to wait physically in long queues without knowing their position or the approximate time required to receive service. Manual queue management causes the following problems:

- Long waiting times for customers.
- Poor queue organisation.
- Difficulty in tracking customer tokens.
- Inefficient staff coordination.
- Lack of real-time queue information.
- Difficulty in maintaining service records.
- Poor customer experience.

1.5 Project Objectives

The objectives of the project are listed in Table 1.1 together with the evidence available for each objective in the present prototype.

ID	Objective	Status / Evidence
OBJ-01	Digitise the customer registration process	Implemented – Customer_Registration form

ID	Objective	Status / Evidence
OBJ-02	Automatically generate unique queue tokens	Implemented (prototype logic) – Deluge workflow
OBJ-03	Maintain token statuses: Waiting, Serving, Completed, Cancelled	Partially implemented – Waiting assigned automatically; other transitions proposed
OBJ-04	Organise services and departments	Forms created – Services, Departments
OBJ-05	Allow staff to manage queue operations	Proposed / partially implemented – Staff_Management form created; Call Next not confirmed
OBJ-06	Maintain queue activity logs	Implemented for the “Created” activity
OBJ-07	Collect customer feedback	Form created – processing not confirmed
OBJ-08	Display queue information through reports and dashboards	Implemented – Waiting Queue report, All Tokens report, dashboard
OBJ-09	Reduce manual work and improve queue organisation	Partly achieved – registration and token creation automated

Table 1.1: Project Objectives and Current Status

1.6 Scope of the Project

The scope of the project covers the design of ten linked data forms, the customer registration process, automatic token generation, the initial Waiting status, creation of an activity log entry, and the display of queue information through reports and a dashboard. The functionality that has been designed but whose complete implementation and testing are not yet confirmed – the Call Next operation, staff and customer role-based permissions, approval workflow, notifications, feedback processing, daily summary automation and the remaining token status transitions – is described in this report as proposed functionality, partially implemented functionality or future scope, and is not presented as a completed feature.

1.7 Organisation of the Report

Chapter 2 presents the literature survey and system study. Chapter 3 gives the requirement analysis. Chapter 4 describes the system design with architecture, data flow, entity relationship and activity diagrams. Chapter 5 documents the implementation, including line-by-line explanation of the Deluge code. Chapter 6 covers testing and debugging. Chapter 7 discusses deployment, security and governance. Chapter 8 presents the results and discussion. Chapter 9 concludes the report and lists future enhancements. The references and Appendices A to F follow.

CHAPTER 2
LITERATURE SURVEY AND SYSTEM STUDY

2.1 Existing Queue Management Approaches

Existing approaches to queue management range from a simple first-come, first-served line to electronic token dispensers and online appointment platforms. The approach chosen by an organisation depends on its size, budget and the nature of the service. The following sub-sections describe the main categories relevant to this project.

2.2 Manual Queue Management Systems

In a manual system customers stand in a physical line or write their names in a register. Staff call names aloud and tick them off. This approach requires no technology, but it provides no information about waiting time, offers no protection against disputes about order of arrival and keeps no structured record of service. During peak periods the line becomes disorganised and customers cannot leave the premises without losing their place.

2.3 Traditional Token Systems

Traditional token systems issue a printed slip or a numbered card on arrival, and a display or a staff member announces the number being served. Tokens improve fairness, but the numbers are normally issued by a mechanical dispenser or by hand. Statuses such as cancelled or completed are not recorded, lost slips cause confusion and the token data cannot be analysed afterwards.

2.4 Digital Queue Management Systems

Digital queue management systems store tokens in a database and show queue information on screens or mobile devices. Commercial products offer features such as SMS alerts, kiosk token printing, multi-counter routing and analytics. They are, however, often expensive, generic and difficult for a small organisation to adapt to its own services. A low-code application allows an organisation to build a queue system that matches its services and data requirements at a fraction of the effort of conventional development.

2.5 Low-Code Development Platforms

Low-code application platforms provide visual builders for forms, reports and pages, and allow limited scripting for custom logic. They shorten development time because the user interface, data storage and hosting are supplied by the platform. Table 2.1 compares conventional development with low-code development on Zoho Creator.

Criterion	Traditional Pro-Code	Low-Code on Zoho Creator
Development effort	Requires a custom user interface, database, APIs and deployment setup.	Uses visual form, report and page builders with configured workflows.
Data capture	Forms and validation are coded individually.	Forms, fields and lookups are configured visually.
Automation	Application logic and jobs must be coded.	Workflows and Deluge scripts support event-driven automation.
Reports and dashboards	Requires charting and reporting libraries.	Reports and pages are built with platform tools.
Hosting	Infrastructure managed separately.	Platform hosting and publishing are provided.
Customisation	Broad control over architecture and UI.	Bounded by platform capabilities and plan limits.
Maintenance	Code-level maintenance and release discipline.	Routine changes are simpler; scripts and permissions still need testing.

Table 2.1: Low-Code and Traditional Development Comparison

2.6 Introduction to Zoho Creator

Zoho Creator is a low-code application development platform from Zoho Corporation. An application is composed of forms (which define data fields), reports (which display records from forms), pages (which present dashboards and custom layouts) and workflows (which run when a record is added, edited or on a schedule). Custom logic is written in Deluge, Zoho's scripting language. Records are stored in the platform's built-in data storage, and lookup fields connect one form to another to form relational links. The project uses these capabilities exclusively; no external database or programming framework is used.

2.7 Existing System Limitations

- Customers wait physically and cannot see their position in the queue.
- Token numbers may be duplicated, lost or issued out of order.
- Staff coordination depends on verbal communication.
- There is no real-time count of waiting and serving customers.
- Service records are not retained for later review.
- Management reports must be compiled manually and are quickly outdated.

2.8 Proposed System

The proposed system digitises the queue in Zoho Creator. A customer is registered through the Customer_Registration form, which is linked to a service. On submission a workflow generates a token with the prefix "T", creates a record in Token_Management with the status Waiting, and writes a "Created" entry in Queue_Activity_Log. The Waiting Queue report, the All Tokens report and the dashboard present the live queue to staff. Figure 2.1 shows the proposed process; the dashed steps are proposed functionality.

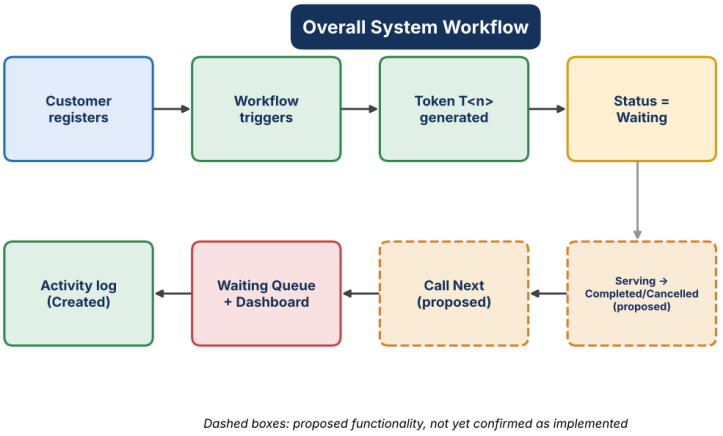


Figure 2.1: *Proposed queue and token process (dashed steps are proposed)*

2.9 Advantages of the Proposed System

- Automatic, consistent token generation without manual numbering.
- Immediate Waiting status for every new token.
- Linked records (service, customer, token, activity) improve traceability.
- Dashboard counts of waiting and serving tokens give a live view of the queue.
- Reduced manual work and improved queue organisation.

CHAPTER 3
REQUIREMENT ANALYSIS

3.1 Functional Requirements

ID	Requirement	Priority	Status
FR-01	Register a customer and link the registration to a service	High	Implemented
FR-02	Generate a unique token with the prefix “T” on registration	High	Implemented (prototype)
FR-03	Set the initial token status to Waiting and record creation time and counter number	High	Implemented
FR-04	Create a Queue_Activity_Log record when a token is created	High	Implemented
FR-05	Display the Waiting Queue and all tokens in reports	High	Implemented
FR-06	Dashboard with waiting count, serving count, Waiting Queue and Currently Serving tables	High	Implemented
FR-07	Call Next: select the oldest Waiting token and change it to Serving	High	Proposed
FR-08	Transition tokens to Completed or Cancelled and log each change	High	Proposed
FR-09	Maintain services, departments and staff records	Medium	Forms created
FR-10	Collect customer feedback	Medium	Form created; processing proposed
FR-11	Send notifications to customers or staff	Medium	Proposed
FR-12	Generate a daily queue summary	Low	Proposed
FR-13	Role-based access for staff, administrators and customers	High	Proposed

Table 3.1: Functional Requirements

3.2 Non-Functional Requirements

ID	Category	Requirement
NFR-01	Usability	The registration form should be simple enough to complete in a few steps on a desktop or mobile browser.
NFR-02	Data integrity	Lookup relationships should keep services, customers, tokens and logs consistent.
NFR-03	Maintainability	Forms, reports and workflows should be modifiable through the platform builders.
NFR-04	Auditability	Token creation and status changes should leave traceable activity records.
NFR-05	Security	Access should be restricted according to user roles (proposed).
NFR-06	Availability	Availability depends on the Zoho Creator service and network connectivity.
NFR-07	Performance	Reports should filter data (for example, by status) to remain responsive.

Table 3.2: Non-Functional Requirements

3.3 Hardware Requirements

Component	Specification
Client device	Desktop, laptop, tablet or smartphone
Processor / RAM	Any device capable of running a modern web browser
Network	Stable internet connection

Component	Specification
Optional display	Monitor or TV to show the queue dashboard at the service counter (proposed use)

Table 3.3: Hardware Requirements

3.4 Software Requirements

Category	Specification
Platform	Zoho Creator (low-code application development platform)
Scripting	Deluge, for custom logic in workflows
Data storage	Zoho Creator built-in application data storage
Interface	Zoho Creator forms, reports, pages and dashboards
Automation	Zoho Creator workflows
Browser	Current version of Google Chrome, Microsoft Edge or Mozilla Firefox

Table 3.4: Software Requirements

No external programming languages, frameworks or databases are used in this project.

3.5 User Roles

Role	Responsibilities
Customer	Registers for a service, receives a token and may provide feedback (feedback proposed).
Staff	Views the Waiting Queue, calls the next token and updates token status (Call Next and updates proposed).
Administrator	Maintains services, departments, staff records and queue settings; reviews reports and the dashboard.

Table 3.5: User Roles and Responsibilities

At the present stage the application has been built and tested mainly from the developer's account. Complete staff and customer role-based permissions have not been confirmed and are listed as proposed.

3.6 Use Cases

Use Case	Actor	Main Flow	Status
Register customer	Customer / Staff	Select a service, enter details and submit the form.	Implemented
Generate token	System	Workflow counts tokens, creates token T<n> with status Waiting and logs the activity.	Implemented
View waiting queue	Staff	Open the Waiting Queue report or dashboard.	Implemented
Call next token	Staff	Select the oldest Waiting token and set it to Serving.	Proposed
Complete / cancel token	Staff	Change the status and log the activity.	Proposed
Submit feedback	Customer	Fill Customer_Feedback after service.	Proposed

Table 3.6: Use Case Descriptions

3.7 Feasibility Study

Technical feasibility: Zoho Creator provides all required building blocks – forms, lookups, Deluge workflows, reports and dashboards – so no additional technology is needed. Economic feasibility: the application is developed on the platform available through the course, and no hardware beyond a browser-capable device is needed. Operational feasibility: the registration form and the queue reports are

simple enough for front-desk staff to use with minimal training. The project is therefore considered feasible as a prototype.

3.8 Functional and Technical Constraints

- The token number is derived from a record count, which is acceptable for a prototype but may produce duplicates (see Section 5.8).
- Call Next and other status transition workflows are not yet confirmed.
- Role-based permissions are not fully configured.
- The application depends on Zoho Creator availability, plan limits and internet access.
- Notifications and feedback processing are not yet implemented and tested.

CHAPTER 4
SYSTEM DESIGN

4.1 System Architecture

The architecture separates the user-facing layer, the automation layer and the stored application records. Customers and staff interact with Zoho Creator forms, reports and the dashboard. Deluge workflows run when a record is submitted and create the token and log records. The built-in data storage holds the ten forms and the lookup links between them, and the reports and dashboard read from these records.

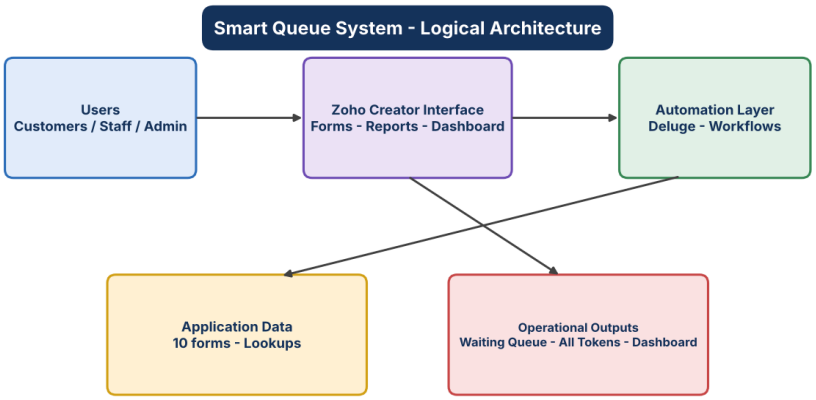


Figure 4.1: *Logical architecture of the Smart Queue and Token Management System*

4.2 Overall System Workflow

The overall workflow starts when a customer is registered. The registration triggers a workflow that generates the token, stores it with the status Waiting and writes an activity log entry. The token then appears in the Waiting Queue report and in the dashboard counts. The later stages – calling the next token, serving, completing or cancelling – are part of the planned queue operation and are shown with dashed outlines in Figure 4.2.

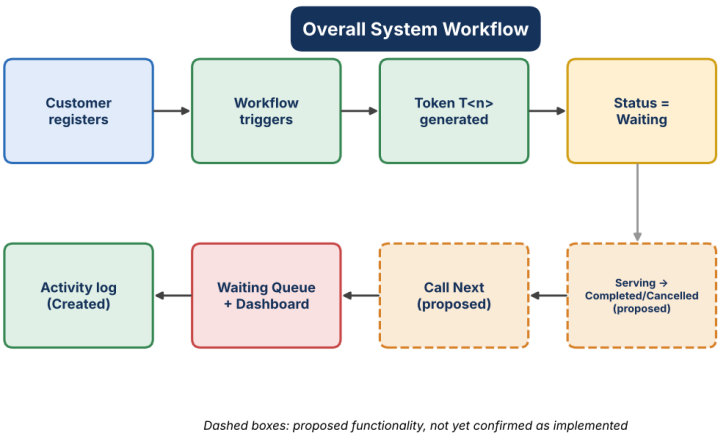


Figure 4.2: *Overall system workflow (dashed boxes are proposed)*

4.3 Module Design

The application consists of ten forms, each acting as a module. Table 4.1 summarises the purpose, expected inputs, processing and outputs of each module. Field details beyond those used in the confirmed Deluge code represent the design intent and should be checked against the final form definitions.

Module	Purpose	Expected inputs	Processing	Outputs / Status
Services	Master list of services offered	Service name, description	Stored as reference data; used by lookup	Service list for registration. Form created
Customer_Registration	Register customers for a service	Customer details, Service (lookup)	On submit, triggers token workflow	Registration record and token. Implemented
Token_Management	Store tokens and statuses	Token_Number, Customer_Registration (lookup), Token_Status, Token_Created_Time, Counter_Number, Added_User	Created by workflow; status to change as queue proceeds	Waiting Queue, All Tokens, dashboard. Implemented (creation)
Queue_Settings	Configuration of queue behaviour	Counter and queue parameters	Read by workflows (planned)	Settings record. Form created
Staff_Management	Maintain staff details	Staff name, contact, assigned counter	Stored as master data	Staff list. Form created
Departments	Organise services by department	Department name, details	Stored as master data	Department list. Form created
Customer_Feedback	Collect feedback after service	Token/customer reference, rating, comments	Stored; analysis planned	Feedback records. Processing not confirmed
Queue_Activity_Log	Audit trail of token events	Token_Management (lookup), Activity_Type, Activity_Description, Activity_Time, Added_User	Created by workflow on token creation	Activity history. Implemented for "Created"
Daily_Queue_Summary	Daily statistics of the queue	Date, totals served/cancelled	Automation planned	Summary records. Not confirmed
Notifications	Messages to customers or staff	Recipient, message, time	Triggered by events (planned)	Notification records. Not confirmed

Table 4.1: Module Design

4.4 Data Flow Diagrams

The context-level diagram (Level 0) shows the system as a single process exchanging data with its two external entities. The Level 1 diagram breaks the system into registration, token generation, activity logging and queue display, and shows the data stores involved.

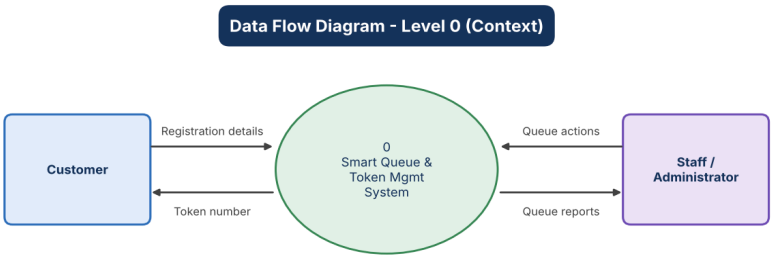


Figure 4.3: *Data Flow Diagram – Level 0*

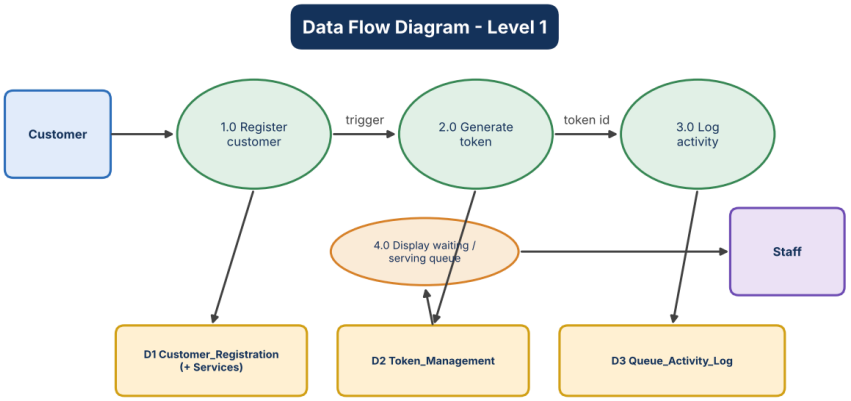


Figure 4.4: *Data Flow Diagram – Level 1*

4.5 Entity Relationship Diagram

Three lookup relationships are confirmed in the application. Customer_Registration contains a Service lookup that links each customer to a record in Services. Token_Management contains a Customer_Registration lookup that links each token to a registration. Queue_Activity_Log contains a Token_Management lookup that links each activity to a token. Each relationship is one-to-many: one service has many registrations, one registration has a token record, and one token has many activity records over its life. The remaining forms are standalone in the confirmed design.

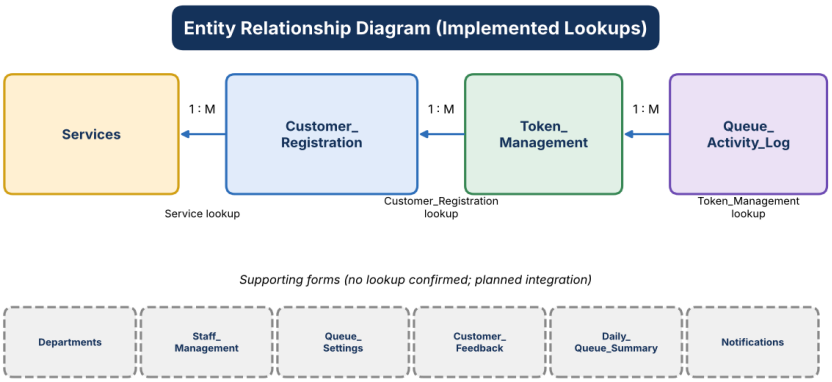


Figure 4.5: *Entity relationship diagram*

Child form	Lookup field	Parent form	Cardinality
Customer_Registration	Service	Services	Many-to-one
Token_Management	Customer_Registration	Customer_Registration	Many-to-one
Queue_Activity_Log	Token_Management	Token_Management	Many-to-one

Table 4.2: Lookup Relationships

4.6 Database and Form Design

Zoho Creator stores the data of each form as records, so the "database" of this application is the set of forms and their lookup links. Table 4.3 lists the fields of the two forms populated by the confirmed workflow.

Form	Field	Value assigned
Token_Management	Token_Number	"T" + next number, e.g. T1

Form	Field	Value assigned
Token_Management	Customer_Registration	ID of the submitted registration
Token_Management	Token_Status	Waiting
Token_Management	Token_Created_Time	Current time (zoho.currenttime)
Token_Management	Counter_Number	1
Token_Management	Added_User	Logged-in user (zoho.loginuser)
Queue_Activity_Log	Token_Management	ID of the new token
Queue_Activity_Log	Activity_Type	Created
Queue_Activity_Log	Activity_Description	Token generated for customer registration
Queue_Activity_Log	Activity_Time	Current time
Queue_Activity_Log	Added_User	Logged-in user

Table 4.3: Fields Populated by the Registration Workflow

Item	Values	Status
Token status	Waiting	Assigned automatically (implemented)
Token status	Serving	Proposed workflow
Token status	Completed	Proposed workflow
Token status	Cancelled	Proposed workflow
Activity type	Created	Implemented
Activity type	Called, Started, Completed, Cancelled	Proposed

Table 4.4: Token Statuses and Activity Types

4.7 Workflow Design

The confirmed workflow is attached to the successful submission of the Customer_Registration form. It performs a fixed sequence of steps without human intervention: count, calculate, insert token, insert log. The proposed workflows – Call Next, status change, notification and daily summary – follow the same pattern: a form or scheduled event triggers a Deluge script that updates the Token_Management status and writes a matching Queue_Activity_Log entry.

4.8 Use Case Diagram

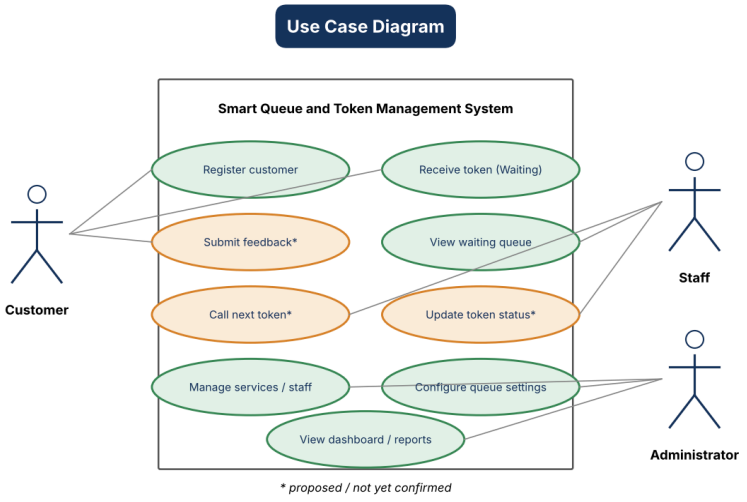


Figure 4.6: *Use case diagram (starred items are proposed)*

4.9 Activity Diagrams

Figure 4.7 shows the activity flow of customer registration, which is implemented. Figure 4.8 shows the lifecycle of a token; only the creation step is confirmed.

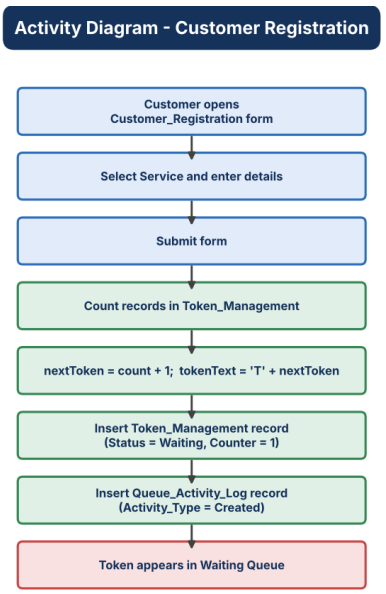


Figure 4.7: *Activity diagram – customer registration*

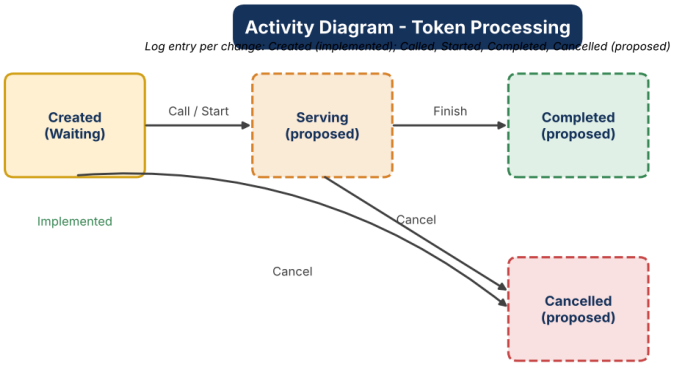


Figure 4.8: *Activity diagram – token processing*

4.10 System Navigation Structure

The application menu groups the ten forms with their reports and the dashboard. Staff use the Waiting Queue report and the dashboard for day-to-day monitoring, while the master forms are used by the administrator.

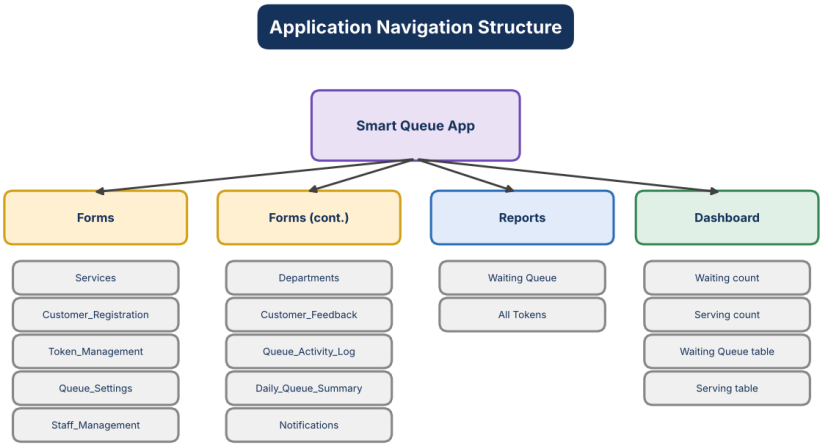


Figure 4.9: *Application navigation structure*

CHAPTER 5
IMPLEMENTATION

5.1 Development Environment

Item	Details
Platform	Zoho Creator
Development approach	Low-code application development
Custom logic	Deluge scripting
Data storage	Zoho Creator built-in storage
Interface	Forms, reports, pages, dashboard
Automation	Zoho Creator workflows
Client	Web browser

Table 5.1: Development Environment

5.2 Zoho Creator Application Development

The application was built using the visual builders of Zoho Creator. Forms were created first, lookup fields were configured to connect them, a workflow was attached to the registration form, and reports and a dashboard page were then built on top of the stored records. Consistent field link names are important because the Deluge script refers to fields such as Token_Number and Token_Status by name.

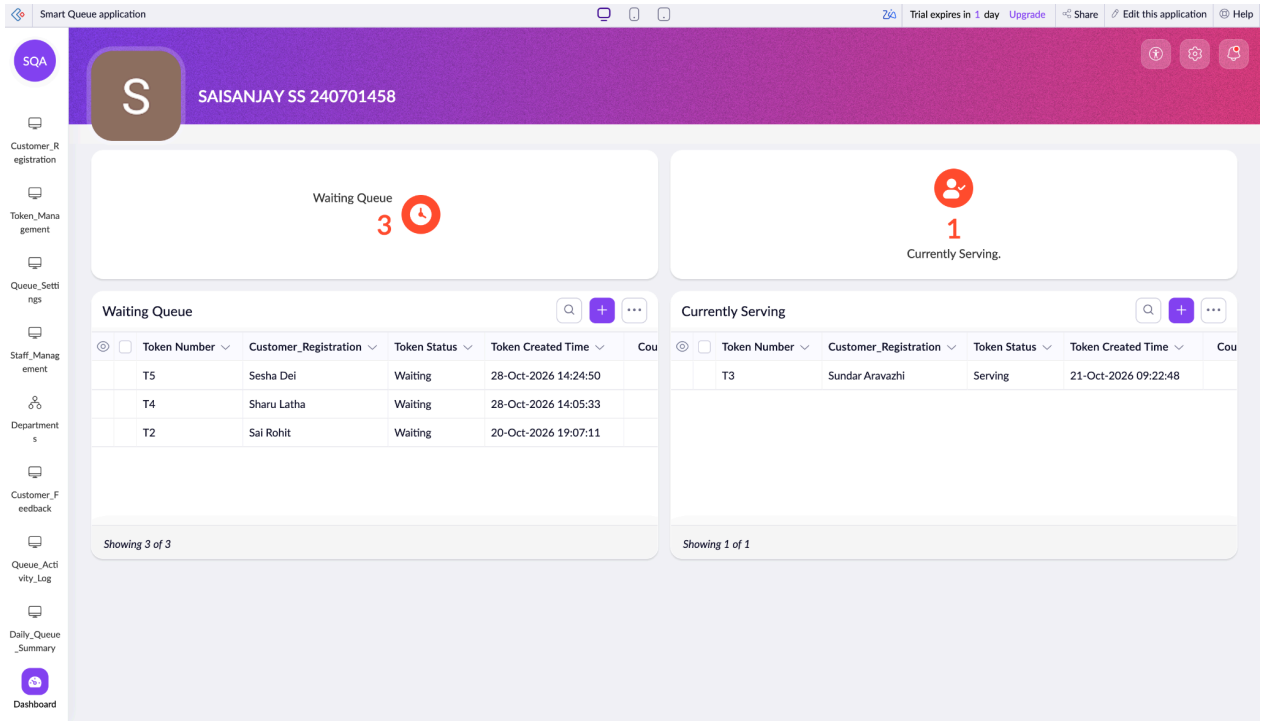


Figure 5.1: *Smart Queue and Token Management application in Zoho Creator*

5.3 Form Creation

Ten forms were created: Services, Customer_Registration, Token_Management, Queue_Settings, Staff_Management, Departments, Customer_Feedback, Queue_Activity_Log, Daily_Queue_Summary and Notifications. Their purposes are described in Table 4.1. Token_Management and Queue_Activity_Log are populated by the workflow rather than typed in by users.

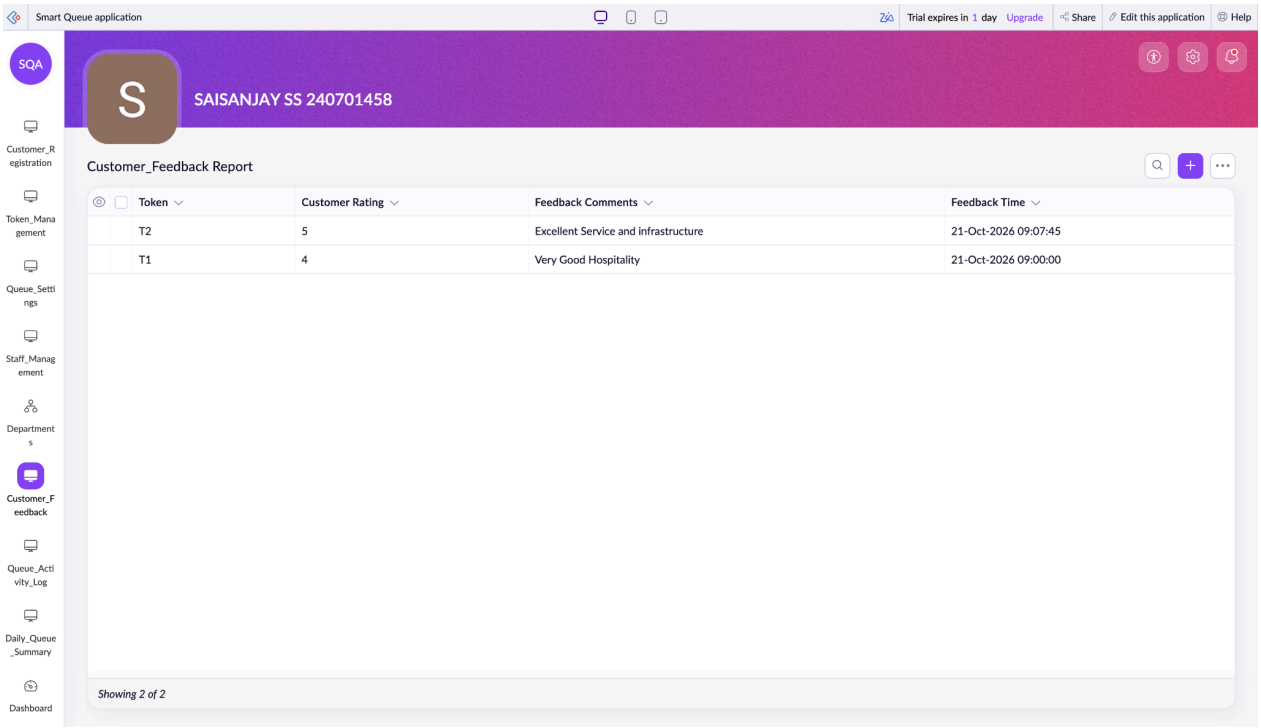


Figure 5.2: *List of application forms*

5.4 Module Implementation

The Customer_Registration module is the entry point of the application. It collects the customer details and the selected service. The Token_Management module stores the token generated for each registration. The Queue_Activity_Log module stores one entry for each token event. The remaining modules (Services, Departments, Staff_Management, Queue_Settings, Customer_Feedback, Daily_Queue_Summary and Notifications) have been created as forms; their workflows are proposed.

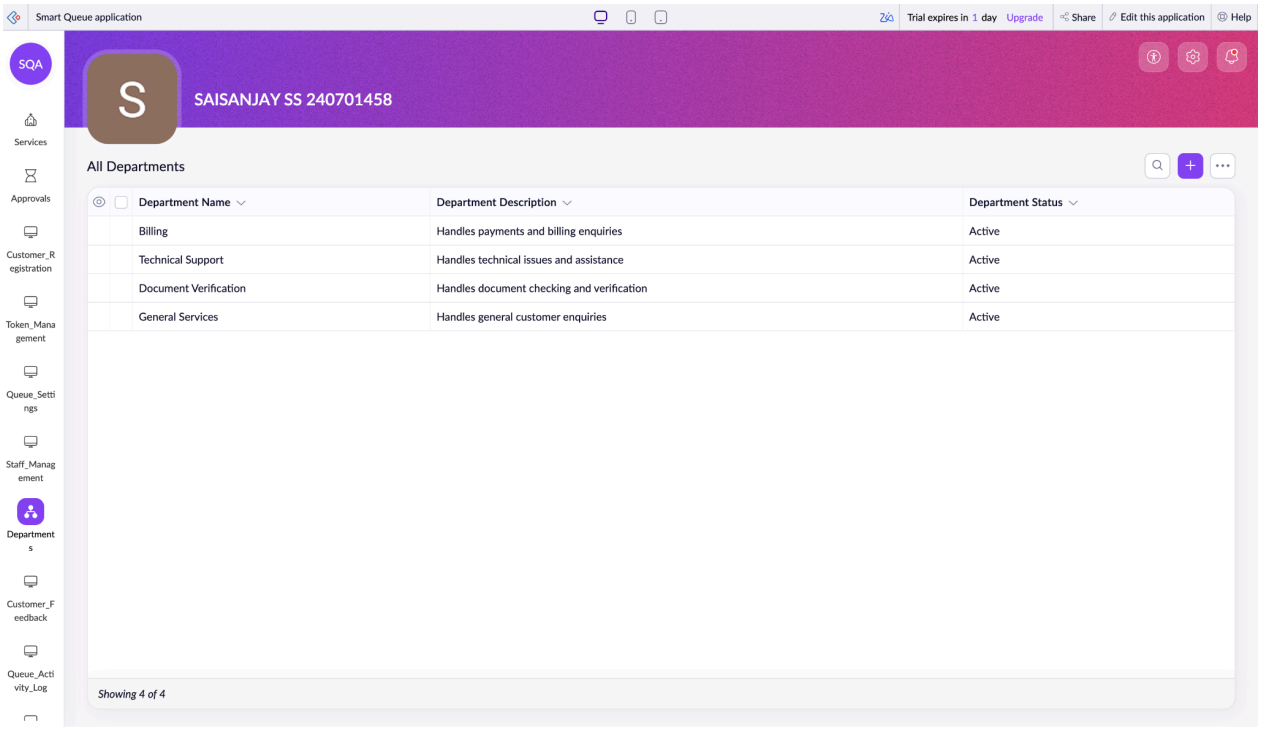


Figure 5.3: *Customer Registration form developed using Zoho Creator*

5.5 Lookup Relationships

A lookup field displays values from another form and stores a link to the selected record. The Service lookup in Customer_Registration lets the user choose from the services defined in Services. The Customer_Registration lookup in Token_Management ties each token to the registration that produced it, and the Token_Management lookup in Queue_Activity_Log ties each log entry to its token. These links allow a user to trace a token back to the customer and service, and to list every event of that token.

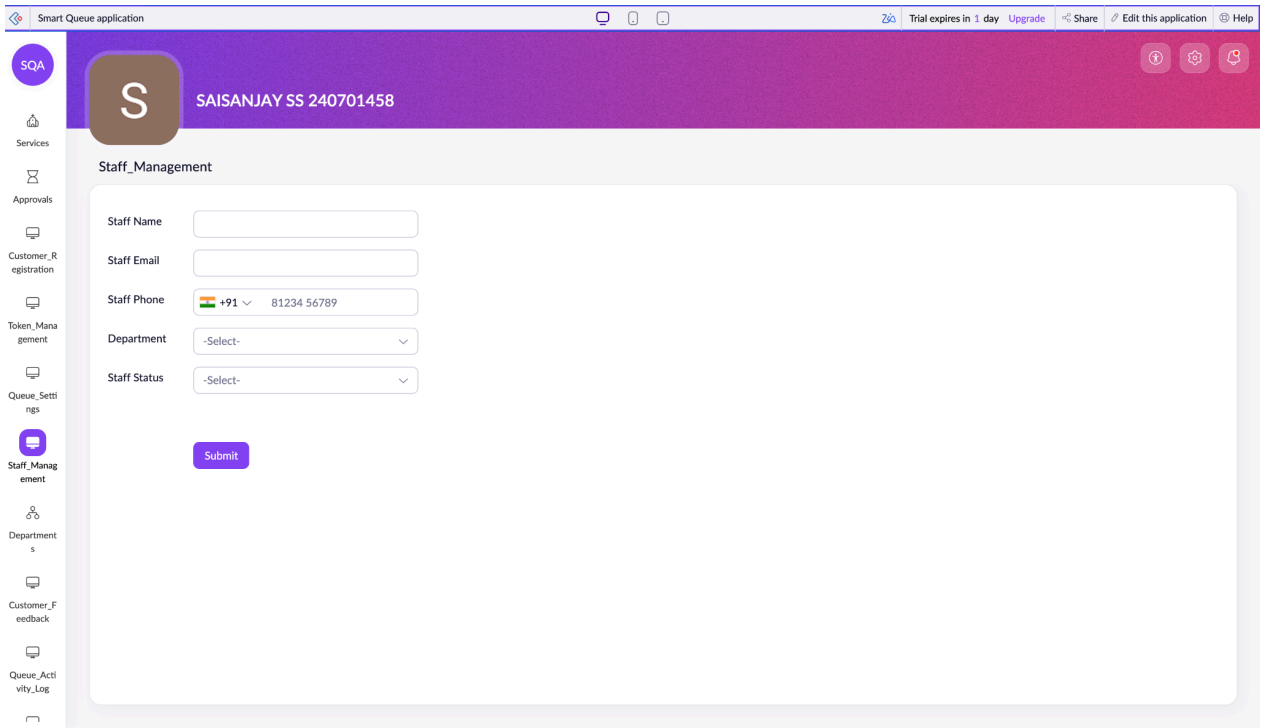


Figure 5.4: *Lookup field configuration*

5.6 Customer Registration Workflow

A workflow is configured on the Customer_Registration form to run when a record is created and the form is successfully submitted. Because it runs after submission, the registration record already exists and has an ID, which the script uses to link the token. The workflow has been implemented and tested. Its steps are:

1. The customer submits the Customer_Registration form.
2. The system counts the existing records in Token_Management.
3. The next token number is calculated and the token text is built with the prefix “T”.
4. A record is created in Token_Management with the status Waiting, the creation time and counter number 1.
5. A corresponding record is created in Queue_Activity_Log.

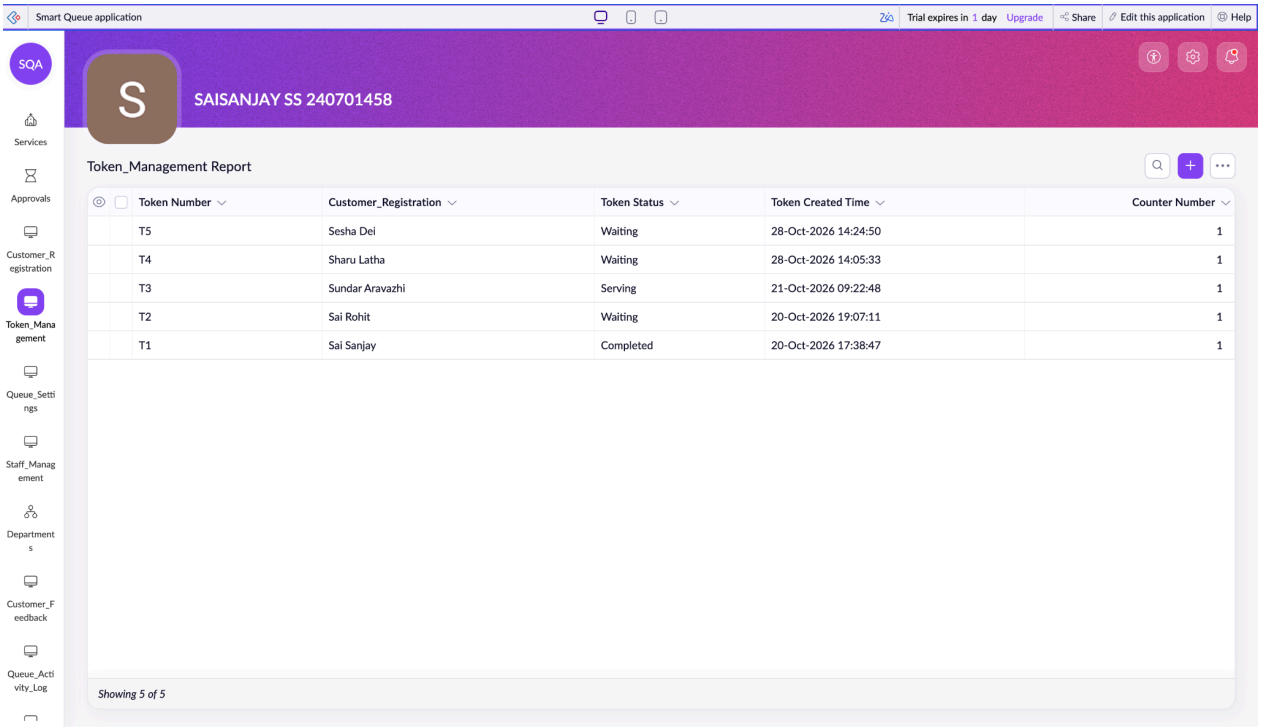


Figure 5.5: *Customer registration workflow configuration*

5.7 Automatic Token Generation

Token generation is fully automatic: the customer and the staff member never type a token number. The script derives the number from the existing token count, converts it to text and prefixes it with “T”, so the tokens follow the pattern T1, T2, T3 and so on in the order of registration. Each token is assigned the status Waiting, the creation time is stamped from the system clock and the default counter number is 1.

5.8 Deluge Code Explanation

The following Deluge code is used in the working customer registration workflow.

```
tokenCount = Token_Management[ID != 0].count();
nextToken = tokenCount + 1;
tokenText = "T" + nextToken.toString();

newTokenID = insert into Token_Management
[
    Token_Number=tokenText
    Customer_Registration=input.ID
    Token_Status="Waiting"
    Token_Created_Time=zoho.currenttime
    Counter_Number=1
    Added_User=zoho.loginuser
];

activityID = insert into Queue_Activity_Log
[
    Token_Management=newTokenID
    Activity_Type="Created"
    Activity_Description="Token generated for customer registration"
    Activity_Time=zoho.currenttime
    Added_User=zoho.loginuser
];
```

Code	Explanation
tokenCount = Token_Management[ID != 0].count();	Fetches all records of Token_Management (the condition ID != 0 is true for every record) and counts them. The result is stored in the variable tokenCount.
nextToken = tokenCount + 1;	Adds one to the count to get the number of the new token.
tokenText = "T" + nextToken.toString();	Converts the number to text and joins it to the letter “T”, for example T1.
newTokenID = insert into Token_Management [...];	Creates a new Token_Management record and stores the ID of the new record in newTokenID.
Token_Number=tokenText	Stores the generated token text.
Customer_Registration=input.ID	Sets the lookup field to the ID of the registration that has just been submitted (input.ID), linking token to customer.
Token_Status="Waiting"	Assigns the initial status.
Token_Created_Time=zoho.currenttime	Stores the current date and time as creation time.
Counter_Number=1	Assigns counter number 1 to the token.
Added_User=zoho.loginuser	Records the user who was logged in when the record was added.
activityID = insert into Queue_Activity_Log [...];	Creates a log record. Token_Management=newTokenID links the log entry to the new token; Activity_Type is “Created”; Activity_Description is a fixed text; Activity_Time and Added_User are filled as above.

Table 5.2: Line-by-Line Explanation of the Deluge Code

Workflow trigger, data insertion and lookup relationship

The script runs on the successful submission of Customer_Registration, so it executes once per new registration. The insert statements add records to other forms directly from Deluge. The lookup fields are filled with record IDs: input.ID for the registration and newTokenID for the token. Because the insert statement returns the ID of the record it creates, the second insert can refer to the first.

Technical limitation of the token-numbering approach

The token number is calculated as record count plus one. This is suitable for a basic prototype, but it has two weaknesses. First, if a Token_Management record is deleted, the count decreases and the next registration may receive a number that already exists. Second, if two customers register at nearly the same moment, both scripts may read the same count and generate the same token number. To make the numbering reliable, a future version should use a platform-generated unique value, such as an Auto Number field or the record ID, or keep a counter in a settings record and validate that the generated token does not already exist. The system also does not currently reset numbering each day.

5.9 Token Status Management

The four statuses used by the application are Waiting, Serving, Completed and Cancelled. Only the initial status, Waiting, is assigned automatically and has been confirmed. The remaining transitions are proposed: Call Next would select the oldest Waiting token (by Token_Created_Time) and change it to Serving; staff would then mark the token Completed after service or Cancelled if the customer leaves. Each transition would write a Called, Started, Completed or Cancelled entry in Queue_Activity_Log.

From	To	Trigger	Status
(new)	Waiting	Customer registration	Implemented
Waiting	Serving	Call Next / Start by staff	Proposed
Serving	Completed	Staff completes service	Proposed
Waiting / Serving	Cancelled	Staff or customer cancels	Proposed

Table 5.3: Token Status Transitions

5.10 Reports and Dashboards

Two reports and one dashboard are implemented. The Waiting Queue report lists tokens whose status is Waiting. The Token Management (All Tokens) report lists every token. The dashboard shows the number of Waiting tokens, the number of Currently Serving tokens, a Waiting Queue table and a Currently Serving table. The Serving count and table will show records only once the Serving status is set by a status-change workflow.

Item	Type	Content
Waiting Queue	Report	Tokens with status Waiting
All Tokens (Token Management)	Report	All token records
Dashboard	Page	Waiting count, Serving count, Waiting Queue table, Currently Serving table

Table 5.4: Reports and Dashboard Implemented

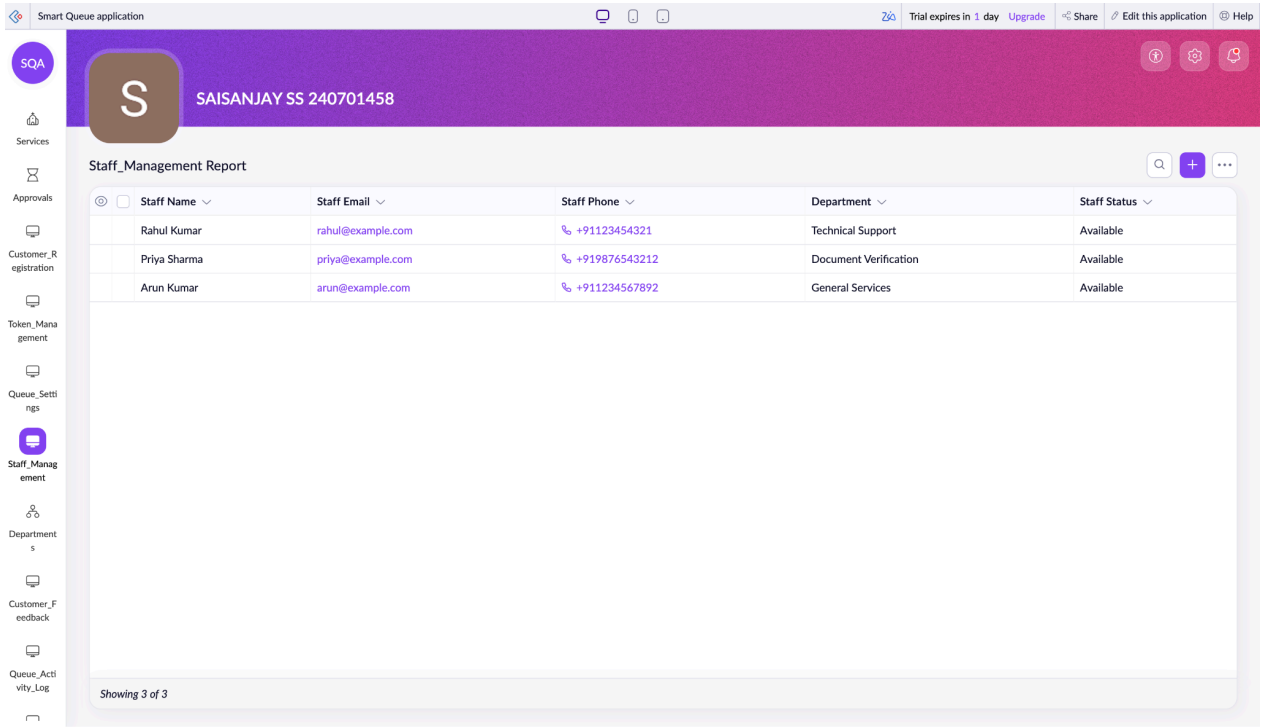


Figure 5.6: *All Tokens report*

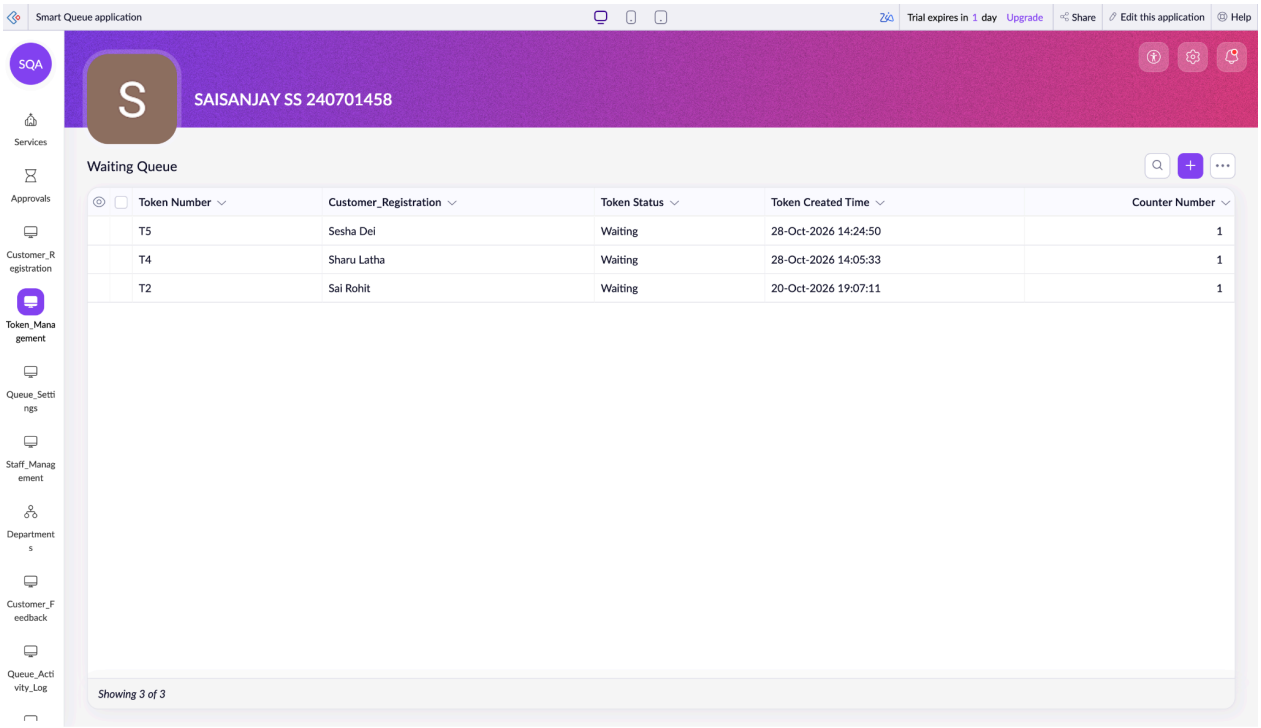


Figure 5.7: *Waiting Queue report*

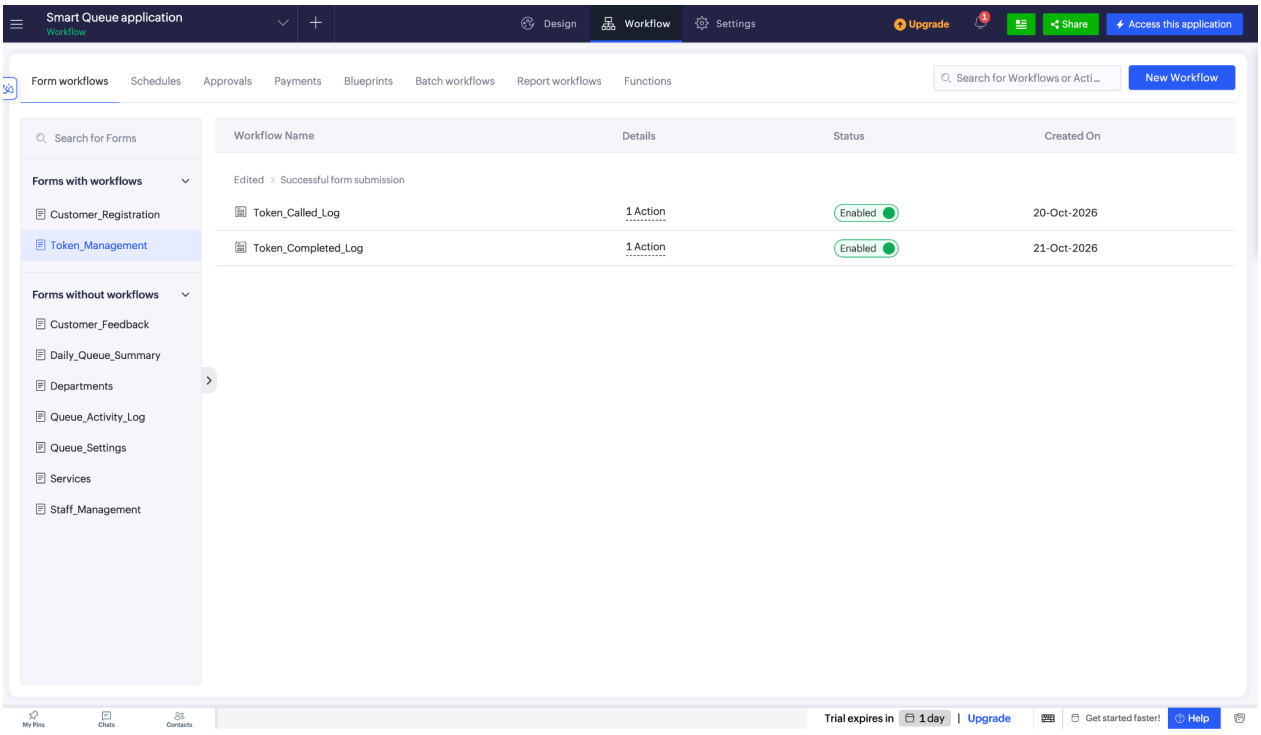


Figure 5.8: *Queue activity log record created during token generation*

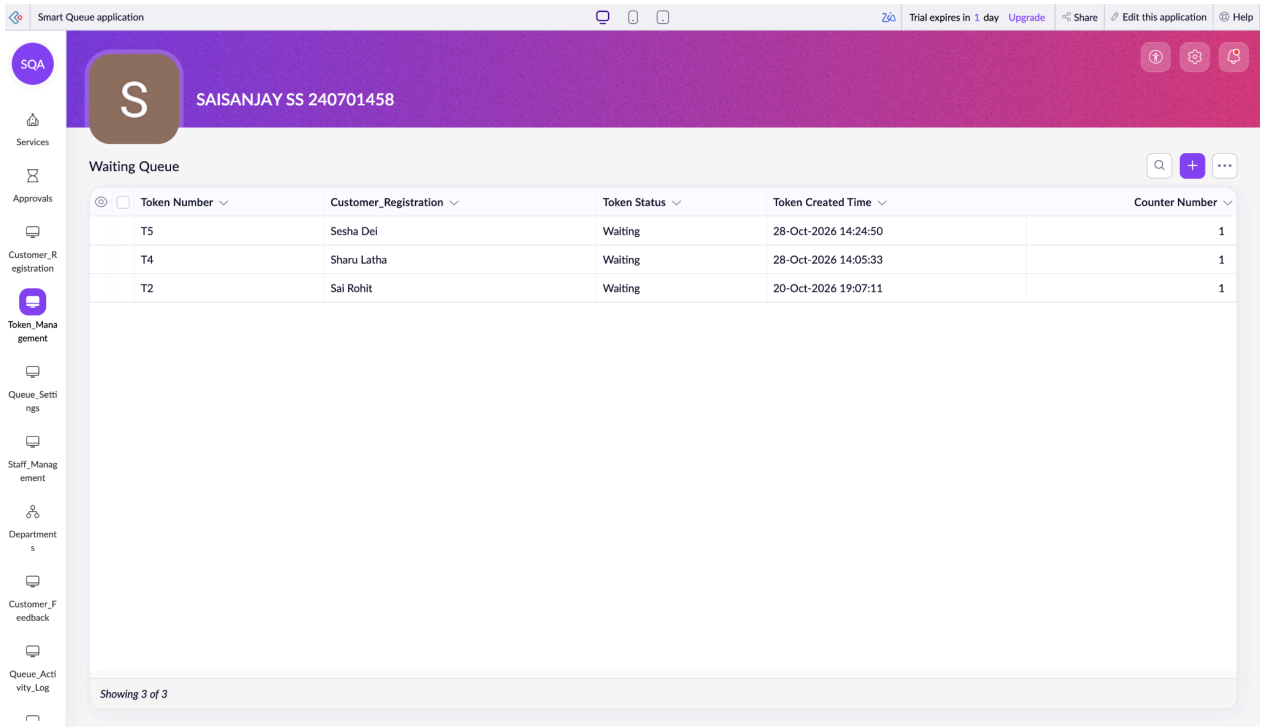


Figure 5.9: *Queue dashboard*

5.11 Workflow Implementation Details

Workflow	Form / Event	Purpose	Status
Token generation	Customer_Registration – on add, on success	Create token and log entry	Implemented and tested
Call Next	Token_Management / button or page	Oldest Waiting token to Serving	Proposed
Status transitions	Token_Management	Serving, Completed, Cancelled	Proposed
Notifications	Token events	Alert customer/staff	Proposed
Feedback processing	Customer_Feedback	Store and analyse feedback	Proposed
Daily summary	Schedule	Fill Daily_Queue_Summary	Proposed

Table 5.5: Workflow Summary

5.12 Screenshots and Explanations

The screenshot placeholders in this chapter and in Chapter 8 must be replaced with captures from the live application. Table 5.6 lists the screen to capture for each figure.

Figure	Screen to capture
Figure 5.1	Application overview in Zoho Creator
Figure 5.2	Design view listing all ten forms
Figure 5.3	Customer_Registration form in live mode
Figure 5.4	Service lookup field properties
Figure 5.5	Workflow builder with the registration workflow
Figure 5.6	All Tokens report with records
Figure 5.7	Waiting Queue report
Figure 5.8	Queue_Activity_Log report
Figure 5.9	Dashboard page

Figure	Screen to capture
Figures 8.1–8.3	Registration submission, generated token, activity log entry

Table 5.6: Screenshot Checklist

CHAPTER 6
TESTING AND DEBUGGING

6.1 Testing Approach

Testing followed a practical approach for a low-code prototype: the workflow was exercised by submitting registrations through the live form and inspecting the resulting records in the reports. Results are reported only for the behaviour that has been confirmed – customer registration and token creation. All other test cases are listed as proposed and are pending validation; no result is claimed for them.

6.2 Unit Testing

Unit testing examined the smallest pieces of logic: the token-number calculation and text construction, the insertion of a Token_Management record with the correct default values, and the insertion of a Queue_Activity_Log record linked to the token. These were checked by submitting a registration and reviewing the generated records.

6.3 Integration Testing

Integration testing checked that the forms work together through lookups: the token record must point to the submitted registration, the activity record must point to the new token, and the Waiting Queue report and dashboard must show the new token. The first three links were confirmed through the working workflow; the dashboard count behaviour with Serving tokens depends on status-change workflows that are pending.

6.4 Functional Testing

Functional testing compared the application behaviour against the requirements in Table 3.1. The registration, token generation, Waiting status, activity logging, reports and dashboard requirements were exercised. Call Next, status transitions, notifications, feedback processing, daily summary and role-based access remain to be tested.

6.5 Test Cases

ID	Test	Expected result	Actual result / Status
TC-01	Submit Customer_Registration form	Registration record saved	Confirmed – demonstrated
TC-02	Token generated after registration	Token with “T” prefix and next number created in Token_Management	Confirmed – demonstrated
TC-03	Initial status of new token	Token_Status = Waiting	Confirmed – demonstrated
TC-04	Token creation time and counter	Token_Created_Time set; Counter_Number = 1	Confirmed – demonstrated
TC-05	Token linked to registration	Customer_Registration lookup filled	Confirmed – demonstrated
TC-06	Activity log entry created	Queue_Activity_Log record with type Created linked to token	Confirmed – demonstrated
TC-07	Waiting Queue report and dashboard	New token appears in Waiting Queue; Waiting count reflects it	Report and dashboard built; verify with live data
TC-08	Call Next selects oldest Waiting token	Oldest token becomes Serving	Proposed – pending validation
TC-09	Complete a Serving token	Status = Completed; log entry	Proposed – pending validation
TC-10	Cancel a token	Status = Cancelled; log entry	Proposed – pending validation
TC-11	Duplicate token numbers after deleting a record	No duplicate numbers	Proposed – known risk with count + 1
TC-12	Two simultaneous registrations	Distinct tokens	Proposed – pending validation
TC-13	Role-based access	Customer cannot access staff operations	Proposed – pending validation

ID	Test	Expected result	Actual result / Status
TC-14	Notifications and feedback	Messages sent; feedback stored	Proposed – pending validation

Table 6.1: Test Cases – Expected and Actual Results

6.6 Error Handling

The current script does not include explicit error handling. If the first insert fails, the second insert would not receive a valid token ID. Planned improvements include checking that newTokenID is not empty before inserting the log entry, validating that the generated token number does not already exist, and showing a clear message to the user when a step fails.

6.7 Debugging

When a workflow does not behave as expected, the developer should confirm the form event and trigger, check field link names such as Token_Status and Customer_Registration, verify lookup values, and review the workflow execution history for the trigger, record and result of each run. Temporary info statements can be used during development and removed afterwards. A common problem in Deluge is a mismatch between a field link name in the script and the name in the form; this produces an error or an empty value.

6.8 Limitations

- Only registration and token creation are confirmed by testing.
- Other test cases are pending and no pass result is claimed.
- Token numbering by record count may produce duplicates.
- No formal user acceptance testing has been carried out.

CHAPTER 7
DEPLOYMENT, SECURITY AND GOVERNANCE

7.1 Application Deployment Using Zoho Creator

Zoho Creator hosts applications on its cloud service, so no separate server or installation is needed. The application is developed in the development environment, and a version can be published to production so that users access it through a link. At the time of writing, the application is a development-stage prototype; a published production URL and deployment record have not been supplied, and the final production deployment remains to be verified.

Check	Verification
Data model	All ten forms, field link names and lookups are correct.
Master data	Services, departments and staff records are loaded.
Workflows	Token-generation workflow is enabled and tested with sample data.
Reports and dashboard	Waiting Queue, All Tokens and dashboard show correct data.
Access	Users and permissions are configured for each role.
Backup	Application and data export procedure is defined.
Release	Date, version and known limitations are recorded.

Table 7.1: Deployment Checklist

7.2 User Access Management

Zoho Creator allows users to be invited to an application and assigned permission profiles that determine which forms, reports and pages they can see and which operations (view, add, edit, delete) they can perform. For this system, separate access for customers, staff and administrators is proposed. Complete role-based permissions have not yet been confirmed.

7.3 Data Security

Customer registration data is stored in the Zoho Creator data store, which is protected by the platform's authentication and access controls. The application records the logged-in user in the Added_User field of token and activity records, which supports accountability. Data access should be limited to users who need it, and exported data should be handled carefully.

7.4 Role-Based Access Concepts

Role	Permitted (proposed)	Restricted
Customer	Register, view own token, submit feedback	Cannot change token status or settings
Staff	View queue, call next, update token status, view logs	Cannot change settings or staff master data
Administrator	Manage services, departments, staff, settings; view all reports	Privileged actions to be reviewed

Table 7.2: Proposed Role-Based Access

7.5 Data Validation

Mandatory fields and a lookup-based service selection keep registration data consistent. The token workflow assigns controlled values for the status and activity type, which prevents free-text variations. Additional validation – for example, preventing duplicate registrations or duplicate token numbers – is proposed.

7.6 Workflow Governance

Workflow changes should be made in the development environment, tested with sample records and published only after the field names and lookups are verified. A short change log recording the date and purpose of each workflow change helps maintain control. Because the token script writes records to two forms, any change to field names must be reflected in the script.

7.7 Privacy Considerations

The application stores customer details collected at registration. Only the information needed to provide service should be collected, access should be restricted to authorised users, and data should not be shared without a legitimate purpose. The organisation deploying the system should define a retention period for registration and token records.

7.8 Backup and Maintenance

Zoho Creator allows application and data export, and regular exports should be scheduled as part of maintenance. Maintenance tasks include reviewing reports for unusual data, checking that workflows run without errors, adding new services, and updating staff records.

7.9 Limitations of the Current Prototype

- Not yet deployed to a verified production environment.
- Role-based access not fully configured.
- Token numbering may produce duplicates.
- Status-transition, notification, feedback and summary workflows are not confirmed.
- No formal backup or recovery procedure has been documented.

CHAPTER 8
RESULTS AND DISCUSSION

8.1 Application Output

The implemented part of the application produces the following outputs: a registration record, an automatically generated token, a token record with status Waiting, an activity log record, and queue views in the Waiting Queue report, the All Tokens report and the dashboard. Screenshots from the live application should replace the placeholders below.

8.2 Customer Registration Output

When the customer submits the Customer_Registration form, the registration is stored and the token workflow runs without any further action by the user.

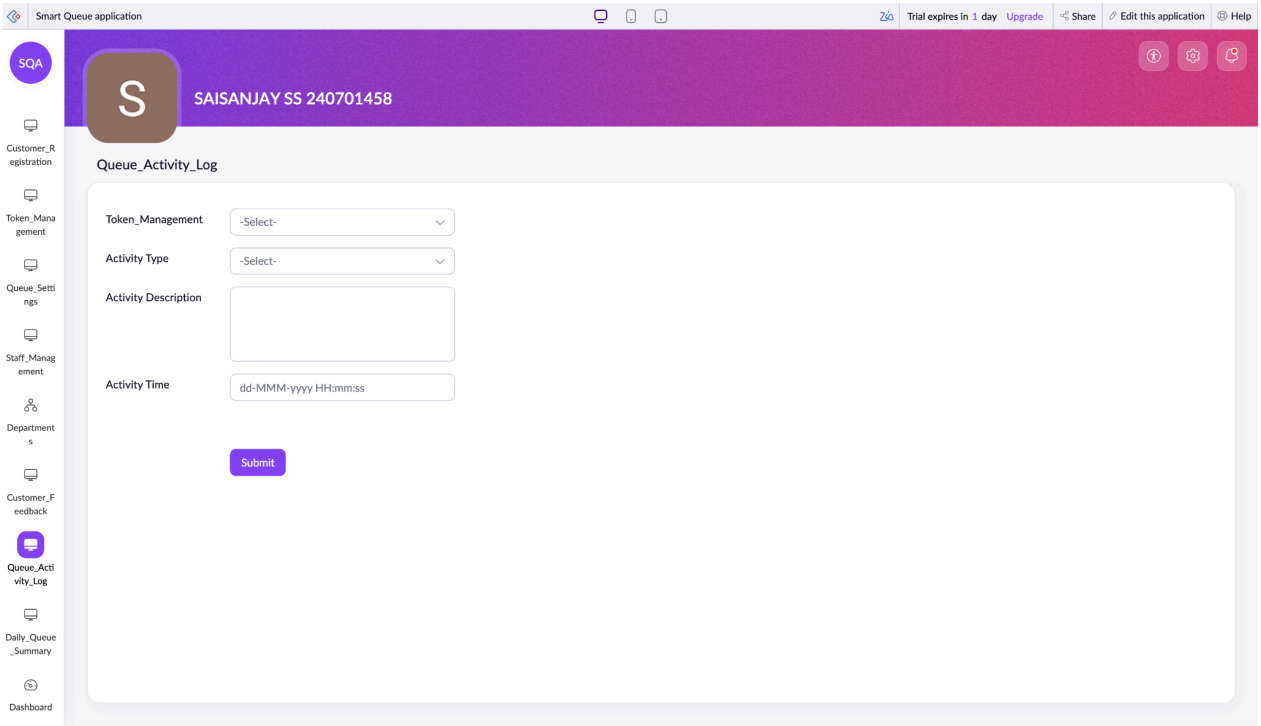


Figure 8.1: *Customer registration submission*

8.3 Generated Token Output

A new record appears in Token_Management with a token number of the form T<n>, the status Waiting, the creation time, counter number 1 and the user who created it. A matching Created entry appears in Queue_Activity_Log.

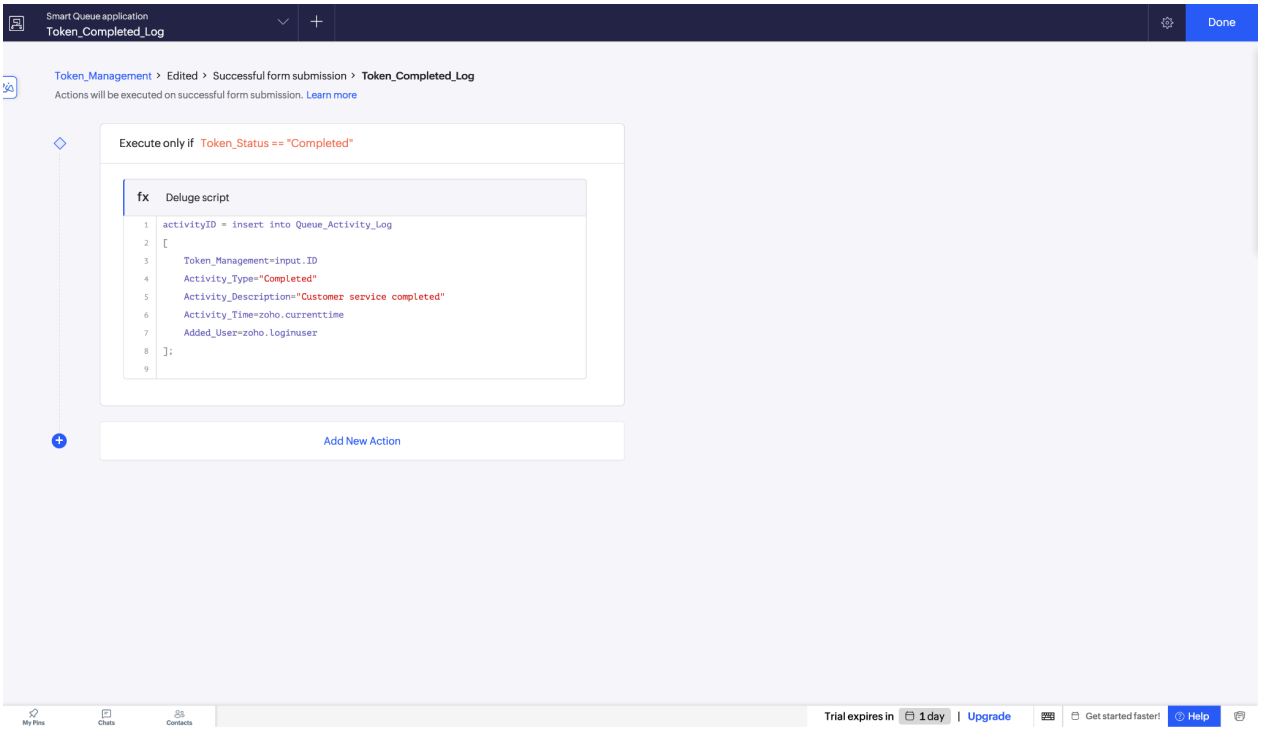


Figure 8.2: *Generated token record*

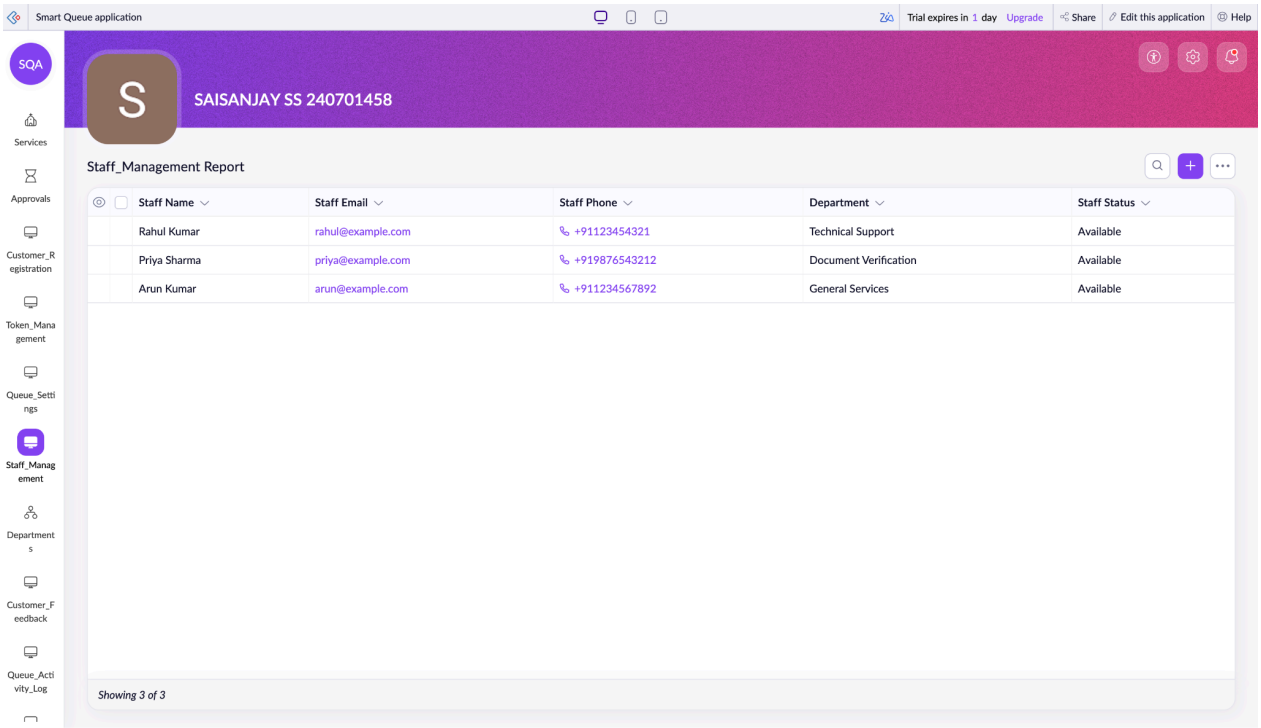


Figure 8.3: *Activity log entry for the new token*

8.4 Queue Management Reports

The Waiting Queue report lists tokens in the Waiting state so that staff can see who is waiting. The All Tokens report is the complete token register and can be searched to review history.

8.5 Dashboard Output

The dashboard presents the number of Waiting tokens and Currently Serving tokens and two tables, Waiting Queue and Currently Serving. It gives staff a single screen for monitoring the queue.

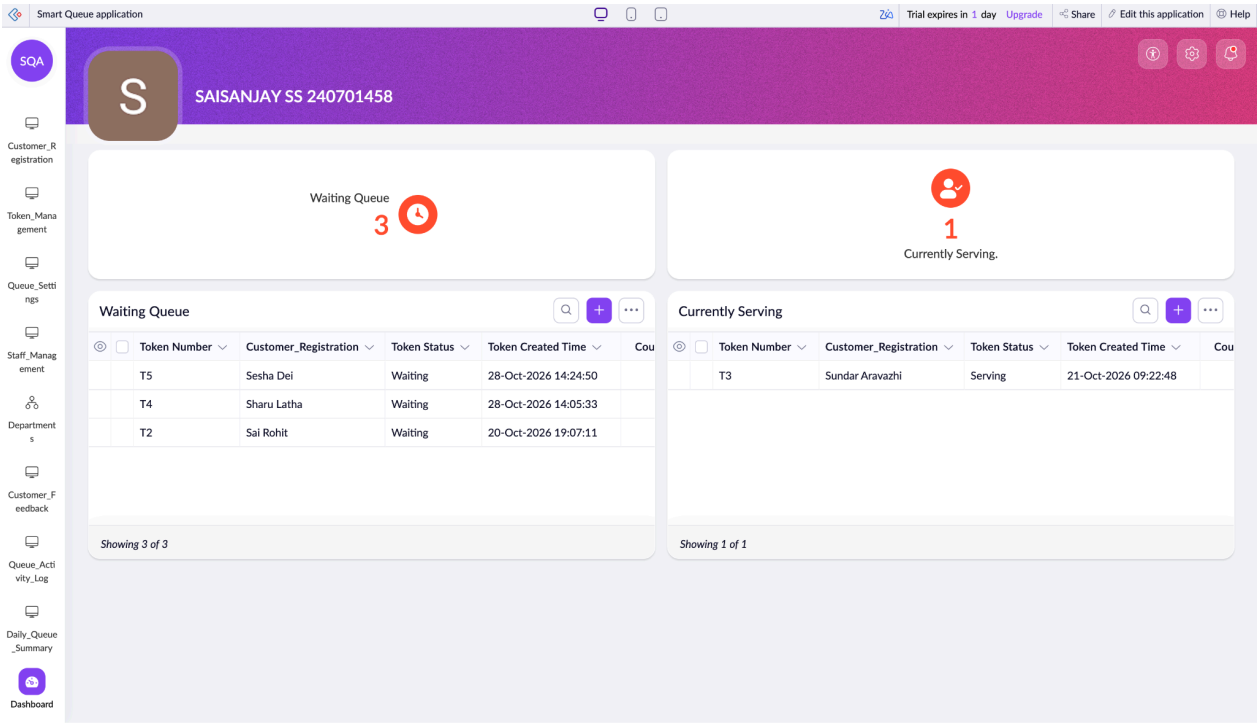


Figure 8.4: *Queue dashboard output*

8.6 Evaluation Against Objectives

Objective	Assessment
Digitise registration	Achieved – form implemented and tested
Automatic unique tokens	Achieved for the prototype; uniqueness not guaranteed in all cases
Token statuses	Partially achieved – Waiting automatic; others proposed
Services and departments	Forms created
Staff queue operations	Proposed – Call Next not confirmed
Activity logs	Achieved for Created events
Customer feedback	Form created; processing proposed
Reports and dashboards	Achieved – two reports and a dashboard
Reduce manual work	Partly achieved – registration and token creation automated

Table 8.1: Objective Achievement Mapping

8.7 Comparison with Traditional Queue Management

Area	Traditional process	Smart Queue prototype
Token issue	Manual slips or register	Automatic “T” token on registration
Status tracking	Not recorded	Token_Status stored (Waiting implemented)
Records	Paper register	Token and activity records in the database
Queue visibility	Physical line	Waiting Queue report and dashboard
Staff coordination	Verbal	Shared digital view; Call Next proposed
Reporting	Manual	Reports and dashboard

Table 8.2: Comparison with Traditional Queue Management

8.8 Benefits

- Less manual effort and fewer numbering errors.

- Immediate, consistent token creation.
- Traceability from service to customer to token to activity.
- Live view of waiting and serving counts.
- Easy modification through low-code builders.

8.9 Current Limitations

- Token numbering depends on record count.
- Call Next and status workflows are not confirmed.
- Role-based permissions and approval workflow are incomplete.
- Notifications, feedback processing and daily summary automation are not confirmed.
- Counter number is fixed at 1.

8.10 Discussion

The project shows that a core part of a queue system – registration, token generation and activity logging – can be built in Zoho Creator with very little code. The ten-form data model provides a structure that can grow into a complete queue management solution. The main work remaining is to implement and test the status-transition workflows and the supporting features, and to make the token numbering robust. The prototype should therefore be regarded as a working foundation, not a finished production system.

CHAPTER 9
CONCLUSION AND FUTURE ENHANCEMENTS

9.1 Project Conclusion

The Smart Queue and Token Management System digitises the first and most important stage of queue handling. A customer who registers receives an automatically generated token that is stored with the status Waiting and logged in an activity record. Queue information is available through reports and a dashboard. The design extends to the complete token lifecycle and to supporting functions, which are documented as proposed functionality.

9.2 Achievements

- Ten linked forms created for the queue domain.
- Customer registration implemented.
- Automatic token generation with the “T” prefix implemented and tested.
- Initial Waiting status assigned automatically.
- Activity log entry created on token creation.
- Waiting Queue and All Tokens reports and a dashboard built.

9.3 Learning Outcomes

The project gave practical experience in form design, lookup relationships, Deluge scripting, workflow triggers, record insertion, reports and dashboards, and in distinguishing implemented features from planned ones when documenting a system.

9.4 Current Limitations

The limitations are the count-based token numbering, the absence of confirmed Call Next and status-change workflows, incomplete role-based permissions and the unconfirmed notification, feedback and summary features (see Section 8.9).

9.5 Future Enhancements

- Implement Call Next to select the oldest Waiting token and set it to Serving.
- Implement Serving, Completed and Cancelled transitions with log entries.
- Replace count + 1 with an Auto Number field or a validated counter.
- Configure role-based permissions for customers, staff and administrators.
- Add notifications when a token is called.
- Process customer feedback and build feedback reports.
- Automate Daily_Queue_Summary with a scheduled workflow.
- Support multiple counters and estimated waiting time.
- Add charts for peak hours and service times.

9.6 Final Remarks

The report documents the system honestly, with confirmed features separated from proposed ones. Completing the proposed workflows would turn the prototype into a fully operational smart queue solution.

REFERENCES

[1] Zoho Corporation, “Zoho Creator Help Documentation,” Zoho. Available: https://www.zoho.com/creator/help/ (accessed Nov. 2026).

[2] Zoho Corporation, “Deluge Help,” Zoho. Available: https://www.zoho.com/deluge/help/ (accessed Nov. 2026).

[3] Zoho Corporation, “Workflow Rules in Zoho Creator,” Zoho Creator Help. Available: https://www.zoho.com/creator/help/ (accessed Nov. 2026).

[4] Zoho Corporation, “Lookup Field in Zoho Creator,” Zoho Creator Help. Available: https://www.zoho.com/creator/help/ (accessed Nov. 2026).

[5] Zoho Corporation, “Reports and Pages in Zoho Creator,” Zoho Creator Help. Available: https://www.zoho.com/creator/help/ (accessed Nov. 2026).

Note: access dates and page-level links should be verified by the student before submission.

APPENDIX A – FUNCTIONAL FEATURE COVERAGE

Feature	Classification	Evidence to attach
Ten application forms	Implemented	Form list screenshot
Customer registration	Implemented	Form and sample record
Automatic token generation	Implemented	Token record
Automatic Waiting status	Implemented	Token record
Token Management records	Implemented	All Tokens report
Queue activity log creation	Implemented	Activity log report
Waiting Queue report	Implemented	Report screenshot
All Tokens report	Implemented	Report screenshot
Dashboard (counts and tables)	Implemented	Dashboard screenshot
Call Next functionality	Proposed	—
Oldest Waiting token to Serving	Proposed	—
Role-based permissions	Proposed	—
Approval workflow	Proposed	—
Notifications	Proposed / not confirmed	—
Customer feedback processing	Proposed / not confirmed	—
Daily queue summary automation	Proposed / not confirmed	—
All status transition workflows	Proposed	—

Table A.1: Feature Coverage

APPENDIX B – APPLICATION FORMS, REPORTS AND DASHBOARD PAGES

B.1 Forms

- Services
- Customer_Registration
- Token_Management
- Queue_Settings
- Staff_Management

- Departments
- Customer_Feedback
- Queue_Activity_Log
- Daily_Queue_Summary
- Notifications

B.2 Reports

- Waiting Queue
- Token Management / All Tokens

B.3 Dashboard

- Waiting token count
- Currently Serving token count
- Waiting Queue table
- Currently Serving table

B.4 Lookup Relationships

Form	Lookup field	Target form
Customer_Registration	Service	Services
Token_Management	Customer_Registration	Customer_Registration
Queue_Activity_Log	Token_Management	Token_Management

Table B.1: Lookup Relationships

APPENDIX C – DELUGE CODE SUMMARY

C.1 Customer registration workflow (form: Customer_Registration, event: on add – on success). The script counts existing tokens, calculates the next number, builds the token text with the prefix "T", inserts a Token_Management record with status Waiting, creation time and counter 1, and then inserts a Created entry in Queue_Activity_Log. The full code and its line-by-line explanation are given in Section 5.8.

Element	Meaning
Token_Management[ID != 0].count()	Fetch and count all token records
.toString()	Convert a number to text
insert into <Form> [...]	Create a record in a form and return its ID
input.ID	ID of the record just submitted
zoho.currenttime	Current date and time
zoho.loginuser	Name of the logged-in user

Table C.1: Key Deluge Elements Used

C.2 Known improvement: the count-based number should be replaced by an Auto Number field or a validated counter to avoid duplicate tokens after deletions or simultaneous submissions.

APPENDIX D – USER MANUAL

D.1 Registering a Customer

6. Open the Smart Queue application in Zoho Creator.
7. Open the Customer_Registration form.

8. Select the required service and enter the customer details.
9. Submit the form.
10. Open Token Management or the Waiting Queue report to see the generated token.

D.2 Viewing the Queue

Open the dashboard to see the Waiting and Currently Serving counts and tables. Open the Waiting Queue report for the list of waiting tokens, or the All Tokens report for all tokens.

D.3 Viewing the Activity Log

Open the Queue_Activity_Log report and locate the entry linked to the token. A newly created token has a Created entry.

D.4 Administrator Tasks

The administrator maintains the Services, Departments, Staff_Management and Queue_Settings forms. Tokens should not be deleted, because deletion can affect the count-based numbering.

D.5 Troubleshooting

Problem	Check
No token generated	Confirm the workflow is enabled and the form was submitted successfully; check execution history.
Token missing from Waiting Queue	Check the Token_Status value and the report filter.
No activity log entry	Check that the second insert statement ran and the lookup is filled.
Duplicate token number	Check whether records were deleted; see the numbering limitation.
Serving count is zero	Serving status is set only by a proposed workflow that is not yet implemented.

Table D.1: Troubleshooting

APPENDIX E – TESTING AND EVALUATION TABLES

Category	Cases	Confirmed	Pending
Registration and token creation	6	6	0
Reports and dashboard	1	Built; verify with live data	—
Status workflows and Call Next	3	0	3
Robustness (duplicates, concurrency)	2	0	2
Access, notifications, feedback	2	0	2

Table E.1: Test Summary

S.No.	Criterion	Max. Marks	Marks Awarded
1	Problem definition and requirement analysis	10	
2	System design	10	
3	Forms, reports and dashboards	15	
4	Deluge scripting and workflows	15	
5	Integration, access control and automation	10	
6	Testing, deployment and security	10	
7	Live demonstration	15	
8	Project report	5	

S.No.	Criterion	Max. Marks	Marks Awarded
9	Viva-voce	10	
	TOTAL	100	

Table E.2: Evaluation Rubric

APPENDIX F – PROJECT EVALUATION / VIVA DOCUMENTATION

Item	Details
Student name	Saisanjay SS
Register number	2116240701458
Project title	Smart Queue and Token Management System
Course	CB23F35 – Low-Code Application Development using Zoho Creator
Project guide	Dr. Jinu Sophia
Date of viva	[SUBMISSION DATE]

Table F.1: Student and Project Details

F.2 Anticipated Viva Questions

Q. No.	Question	Unit	Response(E/G/S/N)	Remarks
1				
2				
3				
4				
5				
6				
7				
8				

Table F.2: Viva Preparation

Component	Max. Marks	Marks Awarded
Criteria 1–8	90	
Viva-voce	10	
TOTAL	100	

Table F.3: Marks Summary

INTERNAL EXAMINER: _____

EXTERNAL EXAMINER: _____